

```
import pandas as pd
import numpy as np
```

```
df = pd.read_csv("weatherAUS.csv", encoding="utf-8", usecols=["Date", "Location", "MinTemp", "MaxTemp"])
```

```
/tmp/ipykernel_5334/2465490239.py:1: DtypeWarning: Columns (2,3) have mixed types. Specify dtype option on import or set low_memory=False.
df = pd.read_csv("weatherAUS.csv", encoding="utf-8", usecols=["Date", "Location", "MinTemp", "MaxTemp"])
```

```
# Drop records where target MinTemp= Nan or MaxTemp=NaN
df = df.dropna(subset=["MinTemp", "MaxTemp"])
```

```
# Convert dates to year-months
df['Year-Month'] = (pd.to_datetime(df['Date'], yearfirst=True)).dt.strftime('%Y-%m')
```

```
-----
ValueError                                Traceback (most recent call last)
/tmp/ipykernel_5334/615352317.py in <cell line: 0>()
      1 # Convert dates to year-months
----> 2 df['Year-Month'] = (pd.to_datetime(df['Date'], yearfirst=True)).dt.strftime('%Y-%m')

-----
      2 frames -----
/usr/local/lib/python3.12/dist-packages/pandas/core/tools/datetimes.py in _array_strptime_with_fallback(arg, name, utc, fmt, exact, errors)
    465     Call array_strptime, with fallback behavior depending on 'errors'.
    466     """
--> 467     result, tz_out = array_strptime(arg, fmt, exact=exact, errors=errors, utc=utc)
    468     if tz_out is not None:
    469         unit = np.datetime_data(result.dtype)[0]

strptime.pyx in pandas._libs.tslibs.strptime.array_strptime()

strptime.pyx in pandas._libs.tslibs.strptime.array_strptime()

strptime.pyx in pandas._libs.tslibs.strptime._parse_with_format()

ValueError: time data "5" doesn't match format "%Y-%m-%d", at position 42290. You might want to try:
- passing 'format' if your strings have a consistent format;
- passing 'format='ISO8601'' if your strings are all ISO8601 but not necessarily in exactly the same format;
- passing 'format='mixed'', and the format will be inferred for each element individually. You might want to use 'dayfirst' alongside this.
```

```
# Derive median daily temperature (mid point between Daily Max and Daily Min)
df['MedTemp'] = df[['MinTemp', 'MaxTemp']].median(axis=1)
```

```
df
```

```
df2 = df[['Location', 'Year-Month', 'MedTemp']].copy()
```

```
# calculate monthly average temperature for each location
df2 = df2.groupby(['Location', 'Year-Month'], as_index=False)['MedTemp'].mean()
```

```
# Transpose dataframe, Pivot to wide format
df2_pivot = df2.pivot(index='Location', columns='Year-Month', values='MedTemp')
```

```
df2_pivot
```

```
# Remove Locations with excessive missing values (NaN)
df2_pivot = df2_pivot.drop(
    ['Dartoon', 'Katherine', 'Melborune', 'Uluru', 'Nhill'], axis=0, errors='ignore')
```

```
# Remove months with lots of missing data (NaN)
df2_pivot=df2_pivot.drop(['2007-11', '2007-12', '2008-01', '2008-02', '2008-03', '2008-04', '2008-05', '2008-06', '2008-07', '2008-08', '2008-09', '200
```

```
df2_pivot
```

```
# add missing months 2011-04, 2012-12, 2013-02 and impute data
df2_pivot['2011-04']=(df2_pivot['2011-03']+df2_pivot['2011-05'])/2
df2_pivot['2012-12']=(df2_pivot['2012-11']+df2_pivot['2013-01'])/2
df2_pivot['2013-02']=(df2_pivot['2013-01']+df2_pivot['2013-03'])/2
```

```
# sort columns so Year-Months are in the correct order
df2_pivot = df2_pivot.reindex(sorted(df2_pivot.columns), axis=1)
```

```
import plotly.graph_objects as go
```

```
from numpy import fill_diagonal
# plot average monthly temperate derived from from daily medians for each location
fig = go.Figure()
for location in df2_pivot.index:
```

```
fig.add_trace(go.Scatter(x=df2_pivot.loc[location, :].index,
                        y=df2_pivot.loc[location, :].values,
                        mode='lines', name=location,
                        opacity=0.8, line=dict(width=1)
                        ))
```

```
# Change chart background color
fig.update_layout(dict(plot_bgcolor = 'white'), showlegend=True)

# Update axes lines
fig.update_xaxes(showgrid=True, gridwidth=1, gridcolor='lightgrey',
                 zeroline=True, zerolinewidth=1, zerolinecolor='lightgrey',
                 showline=True, linewidth=1, linecolor='black',
                 title='Date'
                 )

fig.update_yaxes(showgrid=True, gridwidth=1, gridcolor='lightgrey',
                 zeroline=True, zerolinewidth=1, zerolinecolor='lightgrey',
                 showline=True, linewidth=1, linecolor='black',
                 title='Degrees Celsius'
                 )

# Set figure title
fig.update_layout(title=dict(text="Average Monthly Temperatures", font=dict(color='black')))

fig.show()
```

```
import numpy as np
```

```
def shaping(datain, timestep):
    # Convert input dataframe to array and flatten
    arr = datain.to_numpy().flatten()

    X_list, y_list = [], []
    for i in range(len(datain.columns) - 2 * timestep + 1):
        X_list.append(arr[i:timestep])
        y_list.append(arr[i+timestep:i+2*timestep])

    # Reshape input and target arrays
    X_out = np.array(X_list).reshape(-1, timestep, 1)
    y_out = np.array(y_list).reshape(-1, timestep, 1)
    return X_out, y_out
```

```
#### step 1 - specify parameters ####
timestep=18
location='Canberra'
```

```
##### Step 2 - prepare data
# split data into train & test dataframes
df_train = df2_pivot.iloc[:, :-2*timestep].copy()
df_test = df2_pivot.iloc[:, -2*timestep:].copy()
```

```
# Select one location
dfloc_train = df_train[df_train.index==location].copy()
dfloc_test = df_test[df_test.index==location].copy()
```

```
# Use previously defined shaping function to reshape the data for LSTM
X_train, y_train = shaping(datain=dfloc_train, timestep=timestep)
X_test, y_test = shaping(datain=dfloc_test, timestep=timestep)
```

```
from tensorflow import keras # for building Neural Networks
from keras.models import Sequential # for creating a linear stack of layers for our Neural Network
from keras import Input # for instantiating a keras tensor
from keras.layers import Bidirectional, LSTM, RepeatVector, Dense, TimeDistributed # for creating layers inside the Neural Network
```

```
##### Step 3 - Specify the structure of a Neural Network
model = Sequential(name="LSTM-Model")
# Input Layer - need to specify the shape of inputs
model.add(Input(shape=(X_train.shape[1], X_train.shape[2]), name='Input-Layer'))
# Encoder Layer
model.add(Bidirectional(LSTM(units=32, activation='tanh', recurrent_activation='sigmoid', stateful=False), name='Hidden-LSTM-Encoder-Layer'))
# Repeat Vector
model.add(RepeatVector(y_train.shape[1], name='Repeat-Vector-Layer'))
# Decoder Layer
model.add(Bidirectional(LSTM(units=32, activation='tanh', recurrent_activation='sigmoid', stateful=False, return_sequences=True), name='Hidden-LSTM-Dec'))
# Output Layer, Linear(x) = x
model.add(TimeDistributed(Dense(units=1, activation='linear'), name='Output-Layer'))
```

```
##### Step 4 - Compile the model
model.compile(
    optimizer='adam', loss='mean_squared_error',
    metrics=['mean_squared_error', 'mean_absolute_error']
)
```

```
history = model.fit(X_train, y_train,
                    batch_size=1, epochs=150,
                    validation_split=0.2, shuffle=True, verbose=0)
```

```
##### Step 6 - Use model to make predictions
# Predict results on test data
pred_test = model.predict(X_test)
```

```
##### Step 7 - Print Performance Summary
print("")
print('----- Model Summary -----')
model.summary() # print model summary
print("")
print('----- Weights and Biases -----')
print("Too many parameters to print but you can use the code provided if needed")
print("")
for layer in model.layers:
    print(layer.name)
    for item in layer.get_weights():
        print(" ", item)
print("")
```

```
# Print the last value in the evaluation metrics contained within history file
print('----- Evaluation on Training Data -----')
for item in history.history:
    print("Final", item, ":", history.history[item][-1])
print("")
```

```
# Evaluate the model on the test data using "evaluate"
print('----- Evaluation on Test Data -----')
results = model.evaluate(X_test, y_test)
print("")
```

```
# Plot average monthly temperatures (actual and predicted) for test (out of time) data
fig = go.Figure()

# Trace for actual temperatures
fig.add_trace(go.Scatter(x=np.array(dfloc_test.columns),
                        y=np.array(dfloc_test.values).flatten(),
                        mode='lines',
                        name='Average Monthly Temperatures - Actual (Test)',
                        opacity=0.8,
                        line=dict(color='black', width=1)
                        ))

# Trace for predicted temperatures
fig.add_trace(go.Scatter(x=np.array(dfloc_test.columns[-timestep:]),
                        y=pred_test.flatten(),
                        mode='lines',
                        name='Average Monthly Temperatures - Predicted (Test)',
                        opacity=0.8,
                        line=dict(color='red', width=1)
                        ))

# Change chart background color
fig.update_layout(dict(plot_bgcolor = 'white'))

# Update axes lines
fig.update_xaxes(showgrid=True, gridwidth=1, gridcolor='lightgrey',
                 zeroline=True, zerolinewidth=1, zerolinecolor='lightgrey',
                 showline=True, linewidth=1, linecolor='black',
                 title='Year-Month'
                 )

fig.update_yaxes(showgrid=True, gridwidth=1, gridcolor='lightgrey',
                 zeroline=True, zerolinewidth=1, zerolinecolor='lightgrey',
                 showline=True, linewidth=1, linecolor='black',
                 title='Degrees Celsius'
                 )

# Set figure title
fig.update_layout(title=dict(text="Average Monthly Temperatures", font=dict(color='black')),
                 legend=dict(orientation="h", yanchor="bottom", y=1.02, xanchor="right", x=1)
                 )

fig.show()
```

Start coding or [generate](#) with AI.

